

Reproducing TISER and Exploring Multilingual and Multi-Agent Extensions for Temporal Reasoning

Andrea Di Felice
Politecnico di Torino
s337517@studenti.polito.it

Francesco Mastro Simone
Politecnico di Torino
s348159@studenti.polito.it

Gabriele Merlino
Politecnico di Torino
s343522@studenti.polito.it

Ginevra Muke
Politecnico di Torino
s343516@studenti.polito.it

Salvatore Nocita
Politecnico di Torino
s346378@studenti.polito.it

Abstract—Temporal reasoning remains a challenging capability for language models, particularly in low-resource and cross-lingual settings. In this work, we first reproduce the core findings of the TISER framework using a small 1.5B-parameter model under a controlled setup with stratified data subsets and parameter-efficient fine-tuning via LoRA. Our results confirm that structured multi-stage prompting significantly improves temporal reasoning, especially when aligned with fine-tuning.

Building on this reproduction, we introduce two extensions. First, we evaluate a multilingual setting by translating the dataset and prompt into Italian while preserving the underlying temporal graph structure. We show that structured reasoning transfers across languages under fine-tuning, although performance remains sensitive to prompt–language alignment. Second, we propose an actor–critic inference pipeline that decouples reasoning generation from evaluation, mitigating self-confirmation bias. Our experiments demonstrate that targeted few-shot prompting of the critic yields consistent gains over the reproduced TISER baseline, highlighting the importance of inference-time architectural modularity in temporal reasoning tasks. All code and experimental resources are available on GitHub.

I. INTRODUCTION

Large Language Models have shown strong performance in factual and commonsense question answering, yet temporal reasoning—especially involving event ordering, duration comparison, and multi-step consistency—remains a demanding task. Structured prompting strategies, such as the multi-stage decomposition introduced in TISER [5], aim to address this limitation by explicitly guiding models through reasoning, timeline construction, reflection, and answer generation.

In this work, we first conduct a controlled reproduction of TISER using a compact 1.5B-parameter model. By operating in a low-capacity regime and employing parameter-efficient fine-tuning (LoRA), we assess whether structured prompting remains effective under constrained computational resources. Our reproduction confirms that structured prompting yields substantial performance gains even with a 1.5B-parameter model, indicating that improvements are not exclusively tied to model scale.

We then explore two extensions that probe the robustness of structured temporal reasoning. The first is a multilingual

evaluation in Italian, designed to isolate the effect of language shift while preserving the underlying temporal graph. This allows us to assess whether structured reasoning pipelines generalize beyond English-centric benchmarks. The second extension introduces an actor–critic inference architecture that separates reasoning generation from evaluation. By mitigating acquiescence bias through targeted few-shot prompting, we examine whether inference-time modularity can further enhance temporal consistency.

Together, these investigations provide insight into how prompt structure, language adaptation, and inference architecture interact in shaping temporal reasoning performance in LLMs.

II. REPRODUCTION SETUP

This section describes the experimental setup adopted to reproduce the core findings of the TISER framework. The reproduction is intentionally designed as a controlled baseline for the extensions presented in the second part of the report.

A. Models and Datasets

All experiments are conducted using a compact language model, *Qwen 1.5B Instruct*, in order to assess the effectiveness of structured temporal prompting in a low-capacity regime. We consider both off-the-shelf inference and parameter-efficient fine-tuning via Low-Rank Adaptation (LoRA).

To ensure comparability with subsequent extensions and to limit computational cost we construct fixed stratified subsets corresponding to 10% of the original TISER training split and 10% of the original test split. The sampling procedure preserves the internal composition of the original data, including the distribution across benchmarks and difficulty levels. LoRA fine-tuning is performed on this reduced training subset, while evaluation is carried out on the corresponding test subset.

B. Prompting Strategies

We compare two prompting strategies:

- **Standard Prompt**, in which the model is directly asked to produce an answer without explicit intermediate reasoning or structure, as detailed in Appendix B.

TABLE I
MAIN REPRODUCTION RESULTS ON THE TISER TEST SET (OVERALL SCORES, EXCLUDING SEMANTIC-ONLY SAMPLES).

Model	Training	Prompt	EM	F1
Qwen 1.5B	Off-the-shelf	Standard	0.26	0.36
Qwen 1.5B	Off-the-shelf	TISER	0.35	0.47
Qwen 1.5B	LoRA FT	Standard	0.37	0.42
Qwen 1.5B	LoRA FT	TISER	0.80	0.88

- **TISER Prompt**, which decomposes the task into multiple structured stages—`<reasoning>`, `<timeline>`, `<reflection>`, and `<answer>`—. The full prompt template is reported in Appendix A.

In addition to the complete TISER prompt, we evaluate several ablated variants in which individual stages are removed or isolated (e.g., no reflection, no timeline construction, only reasoning), following the experimental design of the original study.

C. Fine-tuning Protocol and Evaluation

LoRA fine-tuning is applied by training the model to generate the full structured TISER output, including all intermediate stages and the final answer, using a standard next-token prediction objective. During inference, the final prediction is extracted from the `<answer>` tag in the generated output.

Performance is measured using Exact Match (EM) and token-level F1 score. We report overall macro-averaged scores across datasets.

III. REPRODUCTION RESULTS

This section reports the results of the reproduction experiments, comparing standard prompting and TISER prompting under both off-the-shelf and LoRA fine-tuned settings, followed by an ablation study of the TISER components.

A. Standard vs. TISER Prompting

Table I summarizes the main reproduction results. When used in an off-the-shelf setting, the TISER prompt consistently outperforms the standard prompt, highlighting the benefits of structured reasoning even without parameter updates. The performance gap becomes substantially larger after LoRA fine-tuning, where the combination of fine-tuning and TISER prompting yields a dramatic improvement over the standard prompt.

These results confirm the main claim of the original TISER framework: structured prompting substantially improves temporal reasoning performance in a low-capacity regime.

B. Ablation Study

To assess the contribution of individual TISER components, we perform an ablation study using the LoRA fine-tuned model. Results are reported per benchmark and as overall scores in Table II.

The full TISER configuration achieves the best overall performance, though partial variants remain close. Reasoning and timeline construction account for most of the gains over

TABLE II
ABLATION RESULTS USING THE LoRA FINE-TUNED MODEL (EM %).
OVERALL EXCLUDES `TOT_SEMANTIC`.

Setting	L2	L3	TGQA	Easy	Hard	Overall
Only timeline	83.96	53.23	51.42	93.02	85.62	73.84
Only reasoning	81.89	82.73	47.52	93.02	84.69	79.14
No timeline	82.45	84.65	45.39	93.36	86.25	79.73
No reflection	82.08	83.46	47.87	94.68	86.25	79.95
TISER	81.32	85.61	47.16	94.02	88.12	80.32
Standard prompt	30.57	9.35	30.85	66.78	63.12	37.35

the standard prompt, while reflection provides a smaller yet consistent improvement. The “Only timeline” variant shows instability across benchmarks, particularly on L3 and TGQA, indicating that timeline construction alone is insufficient without explicit reasoning. Overall, the results support multi-stage structured prompting, with most benefits driven by reasoning and temporal structuring rather than reflection alone.

IV. MULTILINGUAL EXTENSION: ITALIAN TEMPORAL REASONING

A. Motivation

The original TISER framework evaluates temporal reasoning exclusively in English. More broadly, most temporal question answering benchmarks (e.g., TGQA, TempReason, TimeQA) are English-centric, leaving open the question of whether structured temporal reasoning pipelines such as TISER are robust to language changes. This limitation is particularly relevant for real-world applications, including legal, medical, and historical analysis, where temporal consistency must often be maintained in languages other than English.

Our multilingual extension addresses the following question: *does the TISER prompting and fine-tuning strategy retain its effectiveness when the reasoning language is shifted from English to Italian?* This is not obvious a priori, as linguistic factors such as morphology, surface variability of temporal expressions, and syntactic structure may interact with multi-stage reasoning in non-trivial ways.

B. Dataset Construction and Design Choices

To enable a controlled multilingual evaluation, we construct an Italian version of the TISER dataset by translating the same fixed training and test subsets used in the reproduction experiments. The subsets are kept identical across languages, ensuring that any performance difference is attributable solely to the language shift rather than to variations in data composition or difficulty.

Only natural-language components are translated, namely the question text, the gold answer, and the structured output fields in the training split. The TISER prompt is also translated into Italian while preserving its multi-stage structure. As a result, the multilingual setting differs from the English baseline only at the surface linguistic level, with the reasoning protocol remaining unchanged.

The temporal context is intentionally left in English. Since it consists of a semi-structured serialization of events with explicit identifiers and temporal boundaries, translating it could

TABLE III
OVERALL RESULTS ON THE ITALIAN TISER TEST SET (MACRO-AVERAGE,
EXCLUDING SEMANTIC-ONLY SAMPLES).

Training	Prompt	EM	F1
Off-the-shelf	Standard (IT)	0.18	0.26
Off-the-shelf	TISER (IT)	0.18	0.25
LoRA FT	Standard (IT)	0.18	0.26
LoRA FT	TISER (IT)	0.53	0.62

compromise structural consistency. Keeping it unchanged preserves an identical underlying temporal graph and isolates the effect of language change to the natural-language interface. This design aligns with established cross-lingual QA practices, where a pivot language is maintained for the underlying representation while queries and outputs vary [1], and with multilingual QA over structured knowledge bases that keep symbolic structures fixed across languages [2].

C. Translation Pipeline

The Italian dataset is generated through a controlled, human-in-the-loop translation pipeline based on large language models. Translation is performed using state-of-the-art LLMs under strict constraints on JSON validity, formatting preservation, and structural consistency. The dataset is processed in small chunks of 20 examples to enable incremental validation. Each translated example replaces its English counterpart one-to-one, ensuring exact alignment between source and target splits. This procedure prioritizes temporal fidelity and structural correctness over large-scale automated translation. The exact translation prompt used in all experiments is reported in Appendix C.

D. Experimental Setup

To isolate the effect of language change, we evaluate on the Italian dataset the same model configurations used in the reproduction phase. We consider four settings, obtained by crossing prompting strategy (Standard vs. TISER) and training regime (off-the-shelf vs. LoRA fine-tuned). In the Italian setting, experiments using TISER prompting rely on a fully translated version of the original TISER prompt (Appendix D), which preserves the same multi-stage structure.

English results for the corresponding configurations are taken directly from the reproduction experiments (Table I) and serve as the reference baseline for cross-lingual comparison. Performance is measured using Exact Match (EM) and token-level F1, following the same evaluation protocol adopted throughout the paper.

E. Results on the Italian Dataset

Table III reports the overall performance on the Italian version of the TISER test set, directly comparable with the English reproduction results.

In the off-the-shelf setting, structured prompting does not provide a clear advantage in Italian, with both prompts yielding comparable performance ($EM \approx 0.18$, $F1 \approx 0.25$). Unlike the English reproduction, prompt structure alone does not compensate for a language shift without parameter adaptation.

After LoRA fine-tuning, the Italian TISER prompt achieves $EM = 0.53$ and $F1 = 0.62$. Although still below the English fine-tuned baseline ($EM = 0.80$, $F1 = 0.88$), these results show that structured temporal reasoning can be effectively learned in Italian under parameter-efficient adaptation.

The remaining gap is attributable to the cross-lingual configuration: the temporal context is intentionally kept in English to preserve the underlying event graph, requiring the model to align Italian reasoning with an English semi-structured representation. While this isolates the language variable, it introduces an additional alignment burden that likely increases task difficulty.

When evaluated with the standard prompt, the fine-tuned Italian model drops to off-the-shelf levels ($EM = 0.18$, $F1 = 0.26$), revealing a strong prompt specialization effect. The model internalizes not only reasoning patterns but also the structured TISER format; removing this scaffold disrupts the learned trajectory and neutralizes most fine-tuning gains.

Overall, structured reasoning transfers to Italian under fine-tuning, but its effectiveness depends critically on prompt-language consistency.

V. ACTOR-CRITIC EXTENSION

This section introduces an extension of the original TISER inference pipeline based on an explicit actor-critic decomposition. While the standard TISER framework relies on a single model to generate reasoning, timeline, reflection, and answer, our extension separates these roles at inference time in order to reduce self-confirmation bias, improve robustness, and explore more efficient reasoning architectures.

A. Motivation

In the original TISER setup, the same model is responsible for producing both the reasoning trace and its subsequent self-reflection. Although effective, this design may amplify confirmation bias, as the model tends to validate its own intermediate conclusions [3]. From a system-design perspective, this differs from many real-world reasoning processes, where proposed explanations are reviewed and corrected by an external evaluator [4].

To address this limitation, we introduce a multi-stage actor-critic pipeline in which the generation of reasoning and timeline is decoupled from their evaluation. The goal is to assess whether externalized reflection can improve temporal consistency and reduce reasoning errors.

B. Pipeline Overview

The actor-critic extension introduces two explicit roles: *Actor*, responsible for generating the `<reasoning>` and `<timeline>` stages given a question and its temporal context, and *Critic*, which receives the original input together with the actor’s outputs and produces a reflective assessment of their correctness.

All experiments in this section use the same base model (Qwen 1.5B Instruct), fine-tuned with LoRA on the English TISER training subset. The extension modifies only the inference pipeline, leaving the training procedure unchanged.

C. Dual-Mode Inference: Two-Stage vs. Three-Stage

We consider two inference configurations within the actor-critic framework. In the three-stage setting, inference proceeds sequentially as Actor $\rightarrow$ Critic (reflection) $\rightarrow$ Solver (final answer). The solver receives the critic’s reflection and produces the final answer as a separate generation step.

However, empirical observations suggest that when the critic is instructed to perform a sufficiently thorough evaluation, an additional solver stage often provides limited corrective value. This motivates a simplified two-stage configuration (see Appendix G for the full prompt) in which the critic jointly produces both the `<reflection>` and the final `<answer>` within a single generation step.

The comparison between the two configurations allows us to assess whether deeper sequential decomposition improves temporal reasoning, or whether effective external evaluation alone is sufficient to achieve comparable performance.

D. Anti-Acquiescence Critic via Few-Shot Prompting

A key issue observed during preliminary experiments is a strong acquiescence bias in the critic. Although tasked with evaluation, the critic frequently validates the actor’s reasoning even when subtle temporal inconsistencies are present. This tendency likely arises from architectural and distributional symmetry: actor and critic share the same base model and training data, leading to correlated reasoning patterns.

To mitigate this effect, we introduce a targeted few-shot prompting strategy for the critic (refer to Appendix E), incorporating curated negative examples in which flawed reasoning is explicitly identified and corrected. By exposing the critic to counterexamples, the prompt promotes a more adversarial evaluation behavior.

This design encourages the critic to actively search for inconsistencies rather than defaulting to confirmation, strengthening its role as an external evaluator.

E. Structured Timeline Representation

We revise the timeline representation used in both actor and critic prompts. Instead of a free-form list of events, timelines are expressed as Markdown tables with explicit date–event columns (see Appendix F). This structured format enforces a clearer separation between temporal anchors and event descriptions, reducing ambiguity and making temporal inconsistencies easier to detect during critique.

We evaluate both list-based and table-based representations to assess the impact of intermediate structure on reasoning quality.

The impact of these design choices is evaluated in the following section.

F. Experimental Results and Analysis

Table IV details the performance of our actor-critic configurations compared to the baseline.

TABLE IV
PERFORMANCE COMPARISON. **ST**: STRUCTURED TIMELINE; **FS**: FEW-SHOT. OVERALL SCORES EXCLUDE OOD DATA (`TOT_SEMANTIC`).

Configuration	EM (Overall)	F1	EM (OOD)
Normal (3-Stg)	0.8070	0.8784	0.1829
Normal (2-Stg)	0.8016	0.8738	0.1707
FS (2-Stg)	0.8271	0.8825	0.1789
FS (3-Stg)	0.8076	0.8797	0.1829
ST (2-Stg)	0.8168	0.8852	0.1464
ST (3-Stg)	0.8092	0.8835	0.1992
ST + FS (2-Stg)	0.8233	0.8870	0.1911
ST + FS (3-Stg)	0.8070	0.8779	0.1789

a) *Comparison with the Reproduced Baseline*: Our proposed extensions consistently outperform the strong reproduction baseline (Qwen 1.5B, Table I). While the standard configuration offers marginal gains, the *Few-Shot 2-Stage* variant achieves the top EM score, clearly surpassing the original TISER protocol. Furthermore, combining Few-Shot prompting with Structured Timelines secures the highest F1 score. These results confirm that the performance uplift stems from the enhanced inference pipeline rather than fine-tuning alone.

b) *Two-Stage vs. Three-Stage*: The two-stage setup consistently matches or exceeds the three-stage variant. Once the critic performs a sufficiently strong evaluation, the additional solver step yields no systematic benefit. This suggests that effective external reflection, rather than deeper sequential decomposition, is the primary driver of accuracy.

c) *Effect of Few-Shot Critic*: Few-shot prompting delivers the most consistent improvements, particularly in the two-stage setting. This indicates that exposing the critic to explicit counterexamples effectively mitigates acquiescence bias, enhancing its corrective capacity.

d) *Structured Timeline Representation*: While the tabular representation produces mixed results on in-distribution data, it significantly improves robustness on out-of-distribution tasks. Notably, the three-stage table-based configuration achieves the highest accuracy on the `tot_semantic_test` set (EM OOD), demonstrating superior generalization to unseen temporal structures.

G. Discussion

Overall, separating reasoning generation from evaluation outperforms the TISER baseline, provided the critic is actively guided against inconsistencies. Architectural modularity alone is insufficient; mitigating acquiescence bias is decisive. The *Few-Shot 2-Stage* approach offers the strongest accuracy–efficiency trade-off.

REFERENCES

- [1] A. Asai, J. Kasai, J. H. Clark, K. Lee, and H. Hajishirzi, “Multilingual Extractive Reading Comprehension by Runtime Machine Translation,” *arXiv preprint arXiv:1809.03275*, 2018.
- [2] A. Perevalov, E. Kharlamov, D. Hovland, and M. Giese, “Multilingual Question Answering Systems for Knowledge Graphs—A Survey,” *Semantic Web*, vol. 14, no. 1, pp. 1–32, 2023.
- [3] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, and J. Schulman, “Training Verifiers to Solve Math Word Problems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [4] G. Irving, P. Christiano, and D. Amodei, “AI Safety via Debate,” *arXiv preprint arXiv:1805.00899*, 2018.
- [5] A. Bazaga, R. Blloshmi, B. Byrne, and A. de Gispert, “Learning to Reason Over Time: Timeline Self-Reflection for Improved Temporal Reasoning in Language Models,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, 2025.

APPENDIX A
EXAMPLE OF TISER PROMPT

Example TISER Prompt and Question

You are an AI assistant that uses a Chain of Thought (CoT) approach with reflection to answer queries. Follow these steps:

- 1) Reason through the problem step by step within the `<reasoning>` tags.
- 2) Given your previous reasoning, identify relevant temporal events in the given context for answering the given question within `<timeline>` tags. Assume relations in the context are unidirectional.
- 3) Reflect on your reasoning and the timeline to check for any errors or improvements within the `<reflection>` tags.
- 4) Make any necessary adjustments based on your reflection. If there is additional reasoning required, go back to Step 1, otherwise move to the next step.
- 5) Provide your final, concise answer within the `<answer>` tags. If the answer is a number, just output the number, nothing else. Otherwise, output the entity or event without any additional comments.

Additional Instructions:

- The `<reasoning>`, `<timeline>`, and `<reflection>` sections are for internal reasoning only.
- Do not use enumerations or lists when writing; use plain text such as paragraphs.
- The response to the query must be entirely contained within the `<answer>` tags.

Response Format:

```
<reasoning>
[Your step-by-step reasoning goes here.]
<timeline>
[Relevant temporal events.]
</timeline>
<reflection>
[Reflection on reasoning and timeline.]
</reflection>
[Any adjustments to your thinking.]
</reasoning>
<answer>
[Final answer.]
</answer>
```

Question:

Which event happened in 1990?

Temporal Context:

(Lucas Prescott was born in Northampton, Massachusetts) starts at 1908. (Megan Peterson was born in Harmonyville, Florida) starts at 1921. (Lucas Prescott was married to Megan Peterson) starts at 1946. (Lucas Prescott died in Oceanview) starts at 1990. (Lucas Prescott was married to Megan Peterson) ends at 1990. (Megan Peterson died in Millwood Town, Austin) starts at 1997.

APPENDIX B
EXAMPLE OF STANDARD PROMPT

Example Standard Prompt and Question

You are an AI assistant that answers questions strictly using the provided temporal context. Provide your final, concise answer within the `<answer>` tags. If the answer is a number, output only the number, nothing else. Otherwise, output the entity or event without any additional comments.

Important:

- The response must be entirely contained within the `<answer>` tags.
- Do not use enumerations or lists when writing; use plain text such as paragraphs.

Response Format:

```
<answer>
[Your final answer.]
</answer>
```

Question:

Which event happened in 1990?

Temporal Context:

(Lucas Prescott was born in Northampton, Massachusetts) starts at 1908. (Megan Peterson was born in Harmonyville, Florida) starts at 1921. (Lucas Prescott was married to Megan Peterson) starts at 1946. (Lucas Prescott died in Oceanview) starts at 1990. (Lucas Prescott was married to Megan Peterson) ends at 1990. (Megan Peterson died in Millwood Town, Austin) starts at 1997.

APPENDIX C
EXAMPLE OF TRANSLATION PROMPT FOR THE MULTILINGUAL EXTENSION

Translation Prompt Used for the Italian TISER Dataset

You are translating a dataset used for training and evaluating a temporal reasoning model (TISER). You will receive a JSON ARRAY of examples. Your task is to translate the content from English to Italian.

IMPORTANT CONSTRAINTS (FOLLOW STRICTLY):

- The output **MUST** be a **VALID JSON ARRAY**.
- Use **ONLY** standard ASCII double quotes (").
- Do **NOT** use smart quotes, markdown, or any surrounding text.
- The output must be directly parsable by `json.loads()`.

Structure Preservation:

- Preserve the JSON structure **EXACTLY**.
- Do **NOT** add or remove fields.
- Do **NOT** rename keys.
- Do **NOT** change the order of elements or keys.

Fields to Translate (Mandatory):

- "question"
- "output"
- "answer"

Fields That Must NOT Be Translated:

- Field names, IDs, dataset names
- Dates, years, numbers
- Proper names of people and places
- The "context" field (must remain in English)

Special Tags (Must Be Preserved Exactly):

```
<reasoning>, </reasoning>
<timeline>, </timeline>
<reflection>, </reflection>
<answer>, </answer>
```

Additional Requirements:

- Use fluent Italian.
- Do **NOT** paraphrase, simplify, or summarize.
- Preserve temporal expressions faithfully.
- The content inside `<answer>` tags and the value of the "answer" field must be identical and in Italian.

Output Requirements:

- Return **ONLY** the translated JSON ARRAY.
- No explanations, comments, or additional text.

APPENDIX D
EXAMPLE OF TISER PROMPT (ITALIAN)

Example TISER Prompt and Question (Italian)

Sei un assistente AI che utilizza un approccio di Chain of Thought (CoT) con riflessione per rispondere alle domande. Segui questi passaggi:

- 1) Ragiona sul problema passo dopo passo all'interno dei tag `<reasoning>`.
- 2) Sulla base del ragionamento precedente, identifica gli eventi temporali rilevanti nel contesto fornito per rispondere alla domanda all'interno dei tag `<timeline>`. Assumi che le relazioni nel contesto siano unidirezionali.
- 3) Rifletti sul tuo ragionamento e sulla timeline per verificare eventuali errori o possibili miglioramenti all'interno dei tag `<reflection>`.
- 4) Apporta le modifiche necessarie in base alla riflessione. Se è richiesto ulteriore ragionamento, torna al Passaggio 1; altrimenti, passa al passaggio successivo.
- 5) Fornisci la tua risposta finale e concisa all'interno dei tag `<answer>`. Se la risposta è un numero, riporta solo il numero, senza nient'altro. In caso contrario, riporta l'entità o l'evento senza alcun commento aggiuntivo.

Istruzioni aggiuntive:

- Le sezioni `<reasoning>`, `<timeline>` e `<reflection>` sono destinate esclusivamente al ragionamento interno.
- Non utilizzare enumerazioni o liste durante la scrittura; usa testo semplice sotto forma di paragrafi.
- La risposta alla domanda deve essere interamente contenuta all'interno dei tag `<answer>`.

Formato della risposta:

```
<reasoning>
[Il tuo ragionamento passo dopo passo va qui.]
<timeline>
[Eventi temporali rilevanti.]
</timeline>
<reflection>
[Riflessione sul ragionamento e sulla timeline.]
</reflection>
[Eventuali aggiustamenti al tuo ragionamento.]
</reasoning>
<answer>
[Risposta finale.]
</answer>
```

Question:

Quale evento è avvenuto nel 1990?

Temporal context:

(Lucas Prescott was born in Northampton, Massachusetts) starts at 1908. (Megan Peterson was born in Harmonyville, Florida) starts at 1921. (Lucas Prescott was married to Megan Peterson) starts at 1946. (Lucas Prescott died in Oceanview) starts at 1990. (Lucas Prescott was married to Megan Peterson) ends at 1990. (Megan Peterson died in Millwood Town, Austin) starts at 1997.

APPENDIX E
CRITIC PROMPT WITH FEW-SHOT EXAMPLES

Critic Prompt (Few-Shot)

You are an AI Critic responsible for the evaluation phase of a Chain of Thought process. Your task is to analyze the provided `<reasoning>` and `<timeline>` to identify errors, inconsistencies, or logical flaws based *strictly* on the provided `<context>`.

Follow these steps:

- 1) Read the Question and the Context carefully.
- 2) Analyze the provided "Draft Reasoning" and "Draft Timeline" to check if they accurately reflect the Context.
- 3) Check for specific errors:
 - Hallucinations: Information present in the reasoning but missing from the context.
 - Temporal errors: Incorrect dates or sequence of events.
 - Logical fallacies: Conclusions that do not follow from the premises.
- 4) Provide your evaluation within `<reflection>` tags.
 - If the draft is correct, simply state it.
 - If errors exist, be diagnostic.

Important:

- Do NOT output the answer.
- Do NOT summarize the context or the drafts.
- Your output must contain **ONLY** the `<reflection>` section.
- Do NOT rewrite the reasoning or the timeline.
- Do not use enumerations, use plain text paragraphs.

Here are examples of how you should critique and correct the reasoning:

EXAMPLE 1 (Error Detection)

Question: Who was the president when the treaty was signed?

Temporal Context: The treaty negotiations began in late 1998. The final signature was placed on January 15, 2001. Clinton left office on January 20, 2001, succeeded by Bush.

Draft Reasoning: The treaty was signed in 2001. Bush became president in 2001. Therefore Bush signed it.

Output:

`<reflection>`

The draft reasoning contains a temporal error. The treaty was signed on Jan 15, 2001. Bush took office on Jan 20, 2001. Therefore, Clinton was still in office (by 5 days) when the treaty was signed. The draft incorrectly assumes "2001" automatically means Bush without checking the exact dates.

`</reflection>`

EXAMPLE 2 (Hallucination Detection)

Question: How long did the war last?

Temporal Context: The conflict started in March 1910. A ceasefire was declared in April 1912.

Draft Reasoning: The war started in 1910 and ended in 1914. $1914 - 1910 = 4$ years.

Output:

`<reflection>`

The draft hallucinates the end date. The context explicitly states a ceasefire in "April 1912", not 1914. The draft introduced external knowledge or a random date not present in the source text.

`</reflection>`

Use the following format for your response:

`<reflection>`

[Your critique of the reasoning and timeline, pointing out errors or confirming accuracy based on the context.]

`</reflection>`

Input Data:

Question: {question}

Temporal context: {context}

Draft Reasoning: {draft_reasoning}

Draft Timeline: {draft_timeline}

APPENDIX F
STRUCTURED TIMELINE PROMPT (TABLE VARIANT)

Structured Timeline Prompt (Table Variant)

You are an AI assistant that uses a Chain of Thought (CoT) approach with reflection to answer queries.

Follow these steps:

- 1) Reason through the problem step by step within the `<reasoning>` tags.
- 2) Given your previous reasoning, **identify relevant temporal events and format them as a Markdown table with columns — Date — Event — within `<timeline>` tags.** Assume relations in the context are unidirectional.
- 3) Reflect on your reasoning and the timeline to check for any errors or improvements within the `<reflection>` tags.
- 4) Make any necessary adjustments based on your reflection. If there is additional reasoning required, go back to Step 1, otherwise move to the next step.
- 5) Provide your final, concise answer within the `<answer>` tags. If the answer is a number, just output the number, nothing else. Otherwise, output the entity or event without any additional comments.

Response Format:

```
<reasoning>
[Your step-by-step reasoning goes here.]
<timeline>
| Date | Event |
| :--- | :--- |
| ...  | ...   |
</timeline>
<reflection>
[Reflection on reasoning and timeline.]
</reflection>
[Any adjustments to your thinking.]
</reasoning>
<answer>
[Final answer.]
</answer>
```

Question: {question}

Temporal Context: {context}

APPENDIX G
TWO-STAGE PROMPT (UNIFIED CRITIC + SOLVER)

Two-Stage Prompt (Critic + Solver)

You are an AI assistant responsible for both critique and solution in a Chain of Thought process. Your task is to analyze the provided `<reasoning>` and `<timeline>`, identify errors or confirm correctness, and then provide the final answer based *strictly* on the provided `<context>`.

Follow these steps:

- 1) Read the Question and the Context carefully.
- 2) Analyze the provided "Draft Reasoning" and "Draft Timeline" to check if they accurately reflect the Context.
- 3) Check for specific errors:
 - Hallucinations: Information present in the reasoning but missing from the context.
 - Temporal errors: Incorrect dates or sequence of events.
 - Logical fallacies: Conclusions that do not follow from the premises.
- 4) Within `<reflection>` tags, provide your critique.
 - If the draft is correct, simply state it and confirm the logic holds.
 - If errors exist, be diagnostic and explain what needs to be corrected.
- 5) Within `<adjustments>` tags, apply your critique to produce the corrected reasoning.
 - If the draft was correct, state that the logic holds and confirm the answer.
 - If errors were found, explain how you are correcting them based on the Context.
- 6) Provide your final, concise answer within the `<answer>` tags.

Important:

- Trust the Context above all else.
- The `<reflection>` and `<adjustments>` sections are for your internal reasoning process.
- **MANDATORY:** You MUST output the `<answer>` tags with the final answer.

Response Format:

```
<reflection>
[Your critique pointing out errors or confirming accuracy.]
</reflection>
<adjustments>
[Your corrected logic incorporating your critique.]
</adjustments>
<answer>
[Your final, concise answer to the query.]
</answer>
```

Input Data:

Question: {question}
Temporal context: {context}
Draft Reasoning: {draft_reasoning}
Draft Timeline: {draft_timeline}

APPENDIX H
EXAMPLE OF STANDARD PROMPT (ITALIAN)

Example Standard Prompt and Question (Italian)

Sei un assistente AI che risponde alle domande utilizzando esclusivamente il contesto temporale fornito. Fornisci la tua risposta finale e concisa all'interno dei tag <answer>. Se la risposta è un numero, riporta solo il numero, senza nient'altro. In caso contrario, riporta l'entità o l'evento senza alcun commento aggiuntivo.

Importante:

- La risposta deve essere interamente contenuta all'interno dei tag <answer>.
- Non utilizzare enumerazioni o liste nello stile di scrittura; usa testo semplice sotto forma di paragrafi.

Formato della risposta:

```
<answer>  
[La tua risposta finale.]  
</answer>
```

Question:

Quale evento è avvenuto nel 1990?

Temporal context:

(Lucas Prescott was born in Northampton, Massachusetts) starts at 1908. (Megan Peterson was born in Harmonyville, Florida) starts at 1921. (Lucas Prescott was married to Megan Peterson) starts at 1946. (Lucas Prescott died in Oceanview) starts at 1990. (Lucas Prescott was married to Megan Peterson) ends at 1990. (Megan Peterson died in Millwood Town, Austin) starts at 1997.